

**Sri Lanka Institute of Information Technology**



# **Homomorphic Encryption**

Assignment 1

Year 2 Semester 1

**Submitted by :- IT23363120**

**Introduction to Cyber Security - IE2022**

B.Sc. (Hons) in Information Technology Specializing in Cyber Security

## Table of Contents

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Abstract .....                                     | 3  |
| 2. Introduction to Homomorphic Encryption .....       | 3  |
| 2.1 Evolution of Homomorphic Encryption .....         | 3  |
| 2.2 Function of Homomorphic function .....            | 5  |
| 2.4 Properties of homomorphic algorithms.....         | 7  |
| 2.5 Subtopics in Homomorphic Encryption.....          | 8  |
| 2.5.1 Partial vs. Fully Homomorphic Encryption .....  | 8  |
| 2.5.2 Partial Homomorphic Encryption (PHE) .....      | 9  |
| 2.5.3 Fully Homomorphic Encryption (FHE) .....        | 13 |
| 2.5.4 Key points for FHE [17].....                    | 19 |
| 2.5.5 Challenges in Implementing FHE:.....            | 21 |
| 2.5.6 Applications of Homomorphic Encryption .....    | 22 |
| 3. Future Developments in Homomorphic Encryption..... | 23 |
| 3.1. Performance Optimization .....                   | 23 |
| 3.2. Hybrid Encryption Models: .....                  | 24 |
| 3.3. Post-Quantum Security .....                      | 24 |
| 3.4. Wider Adoption in Healthcare and Finance: .....  | 25 |
| 4. Conclusion .....                                   | 26 |
| 5. References.....                                    | 29 |

# 1. Abstract

Homomorphic encryption is a major encryption method that permits the operations to be performed on encrypted data without the urge to decrypt the cipher text to plain text first [25][3]. This cryptographic approach has emerged as a noticeable innovation, particularly in the terms of sensitive data handling in cloud computing environments [3]. In Classical encryption methods/models it is a requirement , that the data to be decrypted before processing, which exposing it to potential risks for exploitations. Homomorphic encryption eliminates this vulnerability and minimize the potential exploitations, making it highly relevant for industries where data privacy is important, such as healthcare, finance, and government sectors and many more [1][4]. This report dives into the evolution of homomorphic encryption, its significance in today's cybersecurity sector, and explores the potential future developments which are possible in the field [5]. The findings underscore its critical role in secure cloud computing, data analytics, and blockchain technologies, setting the stage for future innovations in privacy-preserving computations [6].

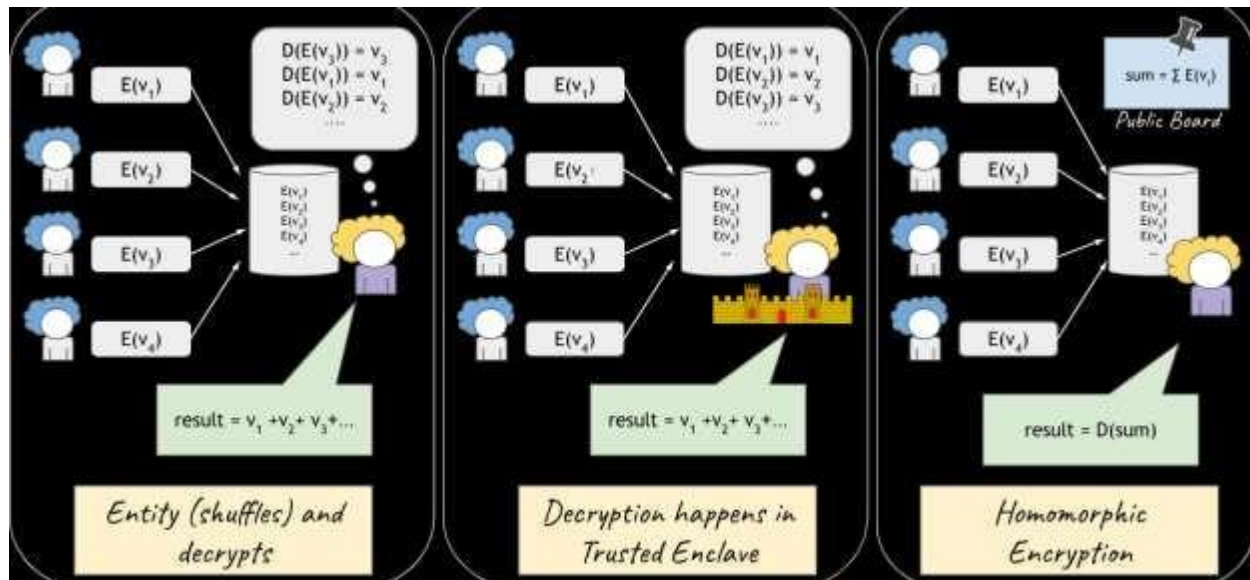
The primordial objective of this report is to show a detailed analysis of homomorphic encryption, its evolution, and its potential applications. Additionally, the report examines the latest research developments and explores the future possibilities for the practical implementation of homomorphic encryption. By reviewing both the theoretical elements and real-world applications, this report presents a comprehensive understanding of the topic.

## 2. Introduction to Homomorphic Encryption

### 2.1 Evolution of Homomorphic Encryption

In the world of Information Technology ,the most common way to store and process data is using third-party/non-primary devices , such as cloud or untrusted computers, service providers ,etc.)[\*].Because of that new problems started to arise.[1] To the discovery of homomorphic encryption , the issue or concern with the legacy encryption schemes , which wont provide the security aspects of the data in a third-party(i.e stores unencrypted data) was involved . Due to

that the 3<sup>rd</sup> party may misuse the data. The Other major concern was to perform any operations on the data , it must be decrypted first.



Represents how HE differ from traditional approaches :-source [1]

The concept of homomorphic encryption goes back to the early 1980s, when the first partial homomorphic schemes were proposed[2]. These early systems, such as RSA, allowed for limited operations on encrypted data, specifically addition or multiplication[1][2][14]. However, a fully homomorphic encryption (FHE) scheme which allowed for computations on encrypted data remained elusive for several decades.[2][17][3]

The breakthrough came in 2009 when Craig Gentry introduced the first practical fully homomorphic encryption scheme[2][16]. Gentry's construction was based on ideal lattices and allowed for both addition and multiplication operations on encrypted data, theoretically enabling any computation to be performed homomorphically [2][4]. However, the initial schemes were far from practical due to the immense computational overhead required, making them inefficient for real-world use.[2][7]

Since Gentry's initial work, there have been noticeable improvements in the field. Several optimized FHE schemes have been proposed, reducing the computational cost and making the technology more viable[2][6]. For example, schemes based on the Learning with Errors (LWE)

(which are very famous in present when developing lattice algorithmic encryptions for post quantum cryptography) problem and Ring-LWE have offered more efficient alternatives and solutions which has then potential for enabling real-world applications in areas such as cloud computing, secure data analytics, and privacy-preserving machine learning.[2][7]

Once these computations are completed, the encrypted result can be decrypted to show the correct, unmodified result of the computation performed on the original data[2][9]. This ability to process encrypted data without first decrypting it is a breakthrough in fields where data privacy and security are essential.[12][2][17][3]

With the exponential growth in cloud computing services and the increasing volume of sensitive data being stored/consumed and processed remotely, the significance of homomorphic encryption is more critical than ever[25][1][3]. Traditional encryption techniques and models, fail to protect data in use, while effectively securing data at rest and in transit [3]. When data is being processed, it must be decrypted, leaving it vulnerable to potential attackers[3].

Homomorphic encryption mitigates this risk by enabling secure data computation without exposing the underlying data.[25]

This paradigm is furthered by cloud computing. Numerous cloud data centers exist, and these retain a massive volume of data from sources and encourage computation that is centered on data. [1] Protecting can be a serious problem for these data centers where data is stored , when the information they stores and process is highly sensitive. Perhaps a data center be targeted, or compromised, and also the capacity for insider assaults. The issues with cybersecurity It is possible to resolve the issue with the outsourced databases if the Crucial information is encrypted. That always results in the issue with the data center's performance analysis using data that is encrypted. [2]

## **2.2 Function of Homomorphic function**

This encryption systems has the ability to perform computational operations to pre encrypted data without knowing the private key, which is a deviation from traditional approach since in traditional approaches , to perform some operations for data , the encrypted data needed to be decrypted first and subsequently perform the needed calculation on the data[13].These calculations performing on encrypted data returns an outcome which is it self going to be

encrypted , ensuring higher security. The results of these possible computations/calculations on the encrypted data is the same as data (raw data) obtained from traditional approaches .[13]

[13]In mathematics (predicate logics) , we said that the system is Homomorphic encryption if ,

***Enc(a) and Enc(b) can perform Enc(f(a, b)); where f can be  
: add, multiplication, XOR, .***

***[ (Enc(a)  $\wedge$  Enc(b))  $\rightarrow$  (Enc(f(a, b))) ; f(x)  $\in$  addition, multiplication, X – or ]***

In cloud computing , when client wants to perform certain calculations or computations on data , the secret key needed to be shared since it will allow access to data from the cloud provider. To solve these issues, homomorphic encryption fits perfectly. The client will allow for could providers to perform calculations on data without decrypting them[13]. The result will be send to the client as encrypted data which will be not an issue ,[13] since to decrypt the data secret key is needed which is only client knows and no one else is able to decrypt neither data nor results in process.[13]

There are 4 general process of the homomorphic encryption systems which is described as follows in study [13].

### **1)Generation of key:**

- The client generates 2 keys :- Secret key  $K_s$  and public key  $K_p$ .

$$(K_s, K_p) = \text{key Gen}(s)$$

**2)Encryption :** an encryption algorithms which takes public key and plain text(m) as inputs and generates a cipher test (C).[n1]

$$C = \text{Enc}_{pk}(m)$$

**3)Evaluation :** applying a function  $f$  to cipher text c using the public key.[13]

$$C \ast = \text{Eval}_{pk}(f, c)$$

**4)Decryption :** a decryption algorithm takes cipher text (c) and secret key and outputs plain text (m)

$$m = Dec_{sk}(c)$$

## 2.4 Properties of homomorphic algorithms.

A proper homomorphic encryption systems must ensures the following properties.[13]

- **Additive Homomorphism (AH) :** The function returns the addition of required data in encrypted form .A homomorphic encryption is additive if [13],

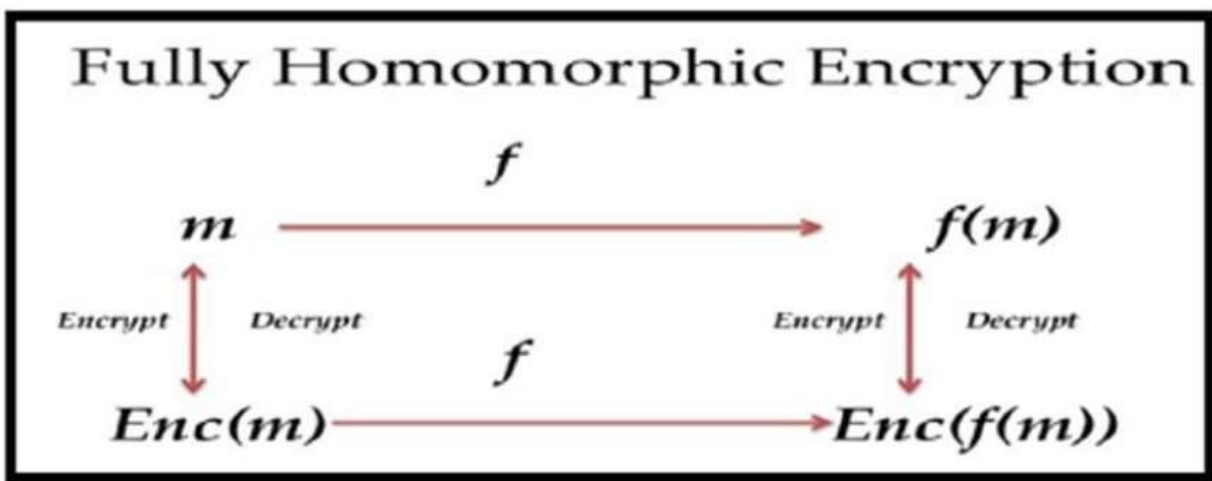
$$Dec_{sk} (Enc_{pk}(m1) + Enc_{pk}(m2)) = m1 + m2$$

- **Multiplicative homomorphism encryption (MH) :** The function returns the multiplication of required data in encrypted form. The homomorphism encryption is multiplicative if[13] ,

$$Dec_{sk} (Enc_{pk}(m1) * Enc_{pk}(m2)) = m1 * m2$$

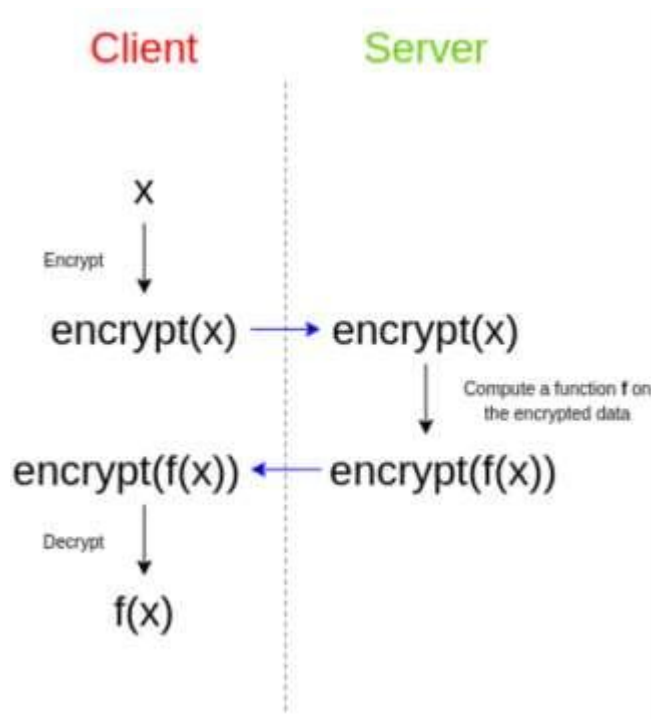
- **Fully Homomorphic :** is said to be called when an algorithm satisfy both additive , multiplicative homomorphism properties simultaneously[13].

Simply the following diagram represents the above process



Source:-A New Encryption Scheme to Perform Smart Computations on Encrypted Cloud Big Data - Scientific Figure on ResearchGate. Available from:

[https://www.researchgate.net/figure/Fully-Homomorphic-Encryption-diagram\\_fig2\\_325155654](https://www.researchgate.net/figure/Fully-Homomorphic-Encryption-diagram_fig2_325155654)



Fully Homomorphic Encryption – an illustration of how it works - Source:

<https://vitalik.ca/general/2020/07/20/homomorphic.html>

## 2.5 Subtopics in Homomorphic Encryption

### 2.5.1 Partial vs. Fully Homomorphic Encryption

- Homomorphic encryption can be divided into two main categories: Partial Homomorphic Encryption (PHE) and Fully Homomorphic Encryption (FHE).

The following tables describes the general comparisons between fully homomorphic encryption and partial homomorphic encryption which was described in studies [2][13][14][1].

| parameter                    | Partial HE                        | Fully HE                         |
|------------------------------|-----------------------------------|----------------------------------|
| Types of operation supported | Either addition or multiplication | Both addition and multiplication |
| Computation                  | Limited number of computations    | Unlimited number of computations |
| Computational efforts        | Less efforts                      | More efforts                     |
| Performance                  | High                              | Low                              |
| Versatile                    | Low                               | High                             |
| Speed                        | High                              | Low                              |
| Ciphertext size              | Small                             | Large                            |
| Examples                     | RSA,Pailler                       | Gentry                           |

### 2.5.2 Partial Homomorphic Encryption (PHE)

PHE schemes allow specific operations, like addition or multiplication, to be performed on encrypted data, but not both. For example:[13][14]

#### **RSA – Multiplicative PHE [13]**

RSA is a very basic algorithm published by Rivest, Shamir and Adelman in 1978 . Despite the fact that its pretty basic algorithm, it is a critical component which based in partial homomorphic encryptions .It supports homomorphic multiplication[12]. You can multiply two ciphertexts, and when decrypted, the result will be the product of the original plaintexts[19].

The large size of keys makes it more secure. But at the very same time its slow in computation due to its complexity and length. The main parameters it used are modulus  $n$  or modular  $n$  , encryption exponent  $e$  and decryption exponent [19][13].The study [13] has described following properties of PHE ,

### 1. Generation of Key Stage

Stage 1: Client creates a private/public keys based on choosing two random large primes number P, Q.

Calculate  $N = P \cdot Q$ , Euler's totient  $\phi(N) = (P-1)(Q-1)$ .

Stage 2: Choose the encryption key E randomly Where,

$1 < E < \phi(N) \mid \gcd(E, \phi(N)) = 1$ .

Stage 3: The two generated pair keys are: the Public key ( $K_p$ )= {E,N} and the Private key ( $K_s$ )= {D,P,Q}

[13]

### 2. Encryption

Stage 1: To be used  $K_p = \{E, N\}$ .

Stage 2: Calculate cipher text  $C = ME \bmod N$ , where  $0 \leq M$ .

[13]

### 3. Decryption

Stage 1: To be used  $K_s = \{D, P, Q\}$ .

Stage 2: Calculate:  $M = CD \bmod N$  to get back the original plain text message

[13]

This homomorphic property of RSA assumes that the two cipher text  $c_1, c_2$ .

$$C_1 = m_1^e \bmod n$$

$$C_2 = m_2^e \bmod n$$

$$C_1 \cdot C_2 = (m_1^e \cdot m_2^e) \bmod n$$

### Mathematical analysis for RSA homomorphism,

Let  $m_1, m_2 \in M$  and  $E(m_1), E(m_2)$  are their corresponding ciphertext :  $c$

$= m^e \bmod n$  where  $(e, n)$  is RSA encryption keys

$$E(m_1) * E(m_2) = [m_1^e \bmod n] * [m_2^e \bmod n] \quad (1)$$

$$= (m_1 * m_2)^e \bmod n \quad (2) = E(m_1 * m_2) \quad (3)$$

$\therefore$  RSA has the multiplicative homomorphism property [16][3][17][13]

### ElGamal encryption

It supports addition, allowing the sum of encrypted data to be computed without decryption. This algorithm was introduced by T.ElGamal in 1984 using probabilistic public-key using discrete logarithms. Even though it is quite similar to Diffie-Hellman concepts [ELGA84, ELGA85], this algorithm has many forms in various standards, including the advanced digital signature standard (DSS) and the S/MIME email standard [1],[14],[13].

Multiplicative homomorphism is presented in this algorithm by the study [13] as follows,

#### Key Generation

1) Select a prime number  $q$ ,

2) Select  $\beta$  where:  $\beta < q$  and  $\beta$  a primitive root of  $q$

2) Select a third prime number  $x$  from  $\{1 \dots q-1\}$  as the private key

4) Calculate  $y$  where:  $y$

$= \beta^x \bmod q$  ( $\beta, q, y$ ) is the public key and  $x$  is the private key

5) The public key is  $(\beta, q, y)$  and the private key is  $(x)$

#### Encrypt

1) Choose a plain text  $M$  where  $M \in \{1, \dots, q\}$

2) Select random number  $k$  where  $k \in \{1, \dots, q\}$

3) Calculate  $K$  where  $K = y^k \bmod q$  4)  $C_1 = \beta^k \bmod q$  5)  $C_2 =$

$KM \bmod q, (C_1, C_2)$  the cipher text

|                                          |
|------------------------------------------|
| <b>Decrypt</b>                           |
| 1) Calculate $K, K = C1x \text{ mod } q$ |
| 2) $M = C2xK - 1 \text{ mod } q$         |

[13]

Analysis of ElGamal homomorphism:

Let  $m1, m2 \in M$  and  $\varepsilon(m1), \varepsilon(m2)$  are their corresponding ciphertext where:

$$\varepsilon(m1) = (c11, c12) = (g^{r1}, m1y^{r1})$$

And

$$\varepsilon(m2) = (c21, c22) = (g^{r2}, m2y^{r2})$$

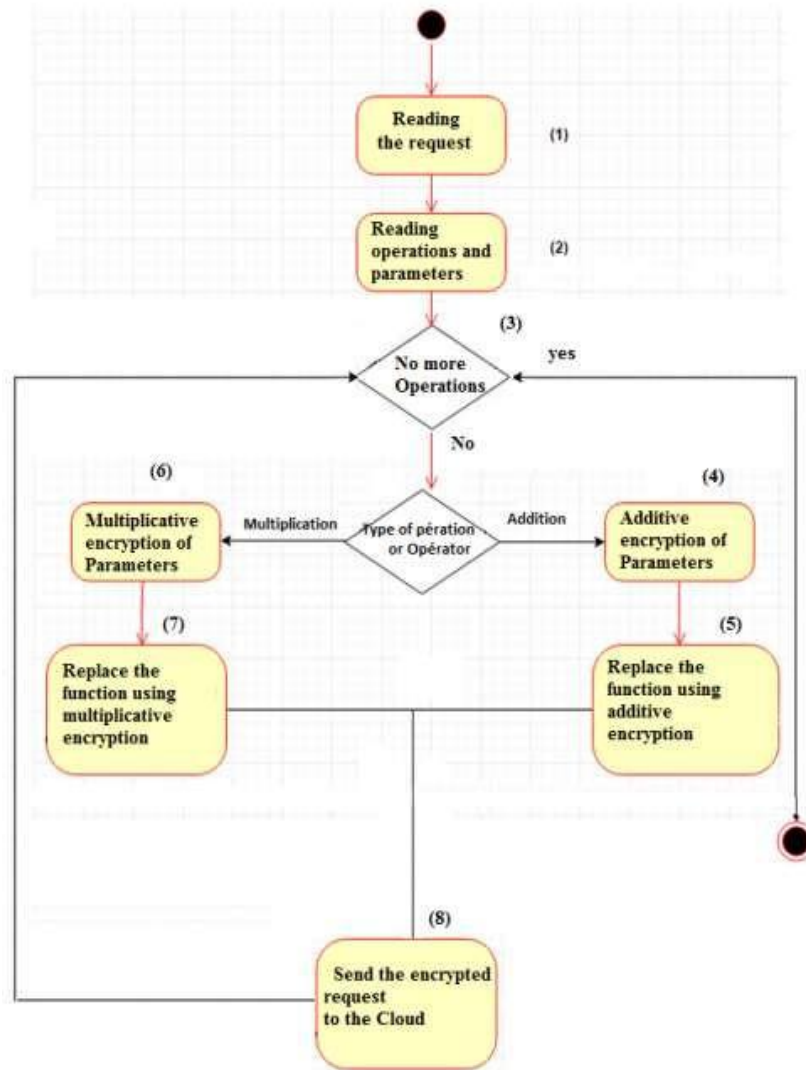
Then:

$$\begin{aligned}\varepsilon(m1) * \varepsilon(m2) &= (c11, c12)(c21, c22) \\ &= (g^{r1}g^{r2}, (m1y^{r1})(m2y^{r2}))(12) \\ &= (g^{r1+r2}, (m1m2)^y)(13) [12]\end{aligned}$$

$\therefore$  ElGamal algorithm achieves the multiplicative homomorphism property [14][16][17][3][13].

- **Use Cases:** PHE is widely and most commonly used in certain applications, like electronic voting, where homomorphic addition helps tally votes while keeping individual choices confidential.

**Following transition diagram shows how partial homomorphic encryption has been used for secure cloud protocol models presented by [20]**



[21]

### 2.5.3 Fully Homomorphic Encryption (FHE)

FHE, introduced by Craig Gentry in 2009, supports both addition and multiplication on encrypted data. This capability allows for **arbitrary computations** on encrypted data, enabling complex functions to be performed without needing to decrypt the data first. It makes FHE much more versatile than PHE, as it can support any computable function while maintaining data privacy. [1]

## Goldwasser-Micali System (GM) – Additive PHE

This is an asymmetric encryption algorithm which was introduced and developed by Shafi Goldwasser and Silvio Micali in 1982 . This can encrypt a single bit eventhough its an additive homomorphic encryption[10],[11],[12].As in study [12] shows following properties ,

### 1. Generation of Key

Stage 1: User generates a public/private key pair by choosing , two large random primes - P, Q.

Stage 2: compute  $N = P.Q$ . Choose  $b \in \mathbb{Z}_N$  |  $b$  is a quadratic non-residue modulo  $n$  and  $(b/n) = -1$

Stage 3: the key pair is obtained as  $(pk, sk)$  Where  $pk = (n, b)$  {public key} And  $sk = (p, q)$  {private key}

[12]

### 2. Encryption

Input message  $M$  is made up of multiple bits  $m_1, m_2, \dots, m_t$

Step 1: For every bit  $m_i$ , a random value  $y_i$  is generated |  $\gcd(y_i, N) = 1$

Step 2: compute:  $C_i = y_i^2 \times m_i \pmod{N}$

[12]

### 3. Decryption Input

message  $C$  is made up of multiple bits  $c_1, c_2, \dots, c_t$

Step 1: calculate:  $e_i = (c_i/p), \forall i \in [1, t]$

Step 2: if  $e_i = 1$  then  $m_i = 0$  else  $m_i = 1$  Output message is  $M = m_1, m_2, \dots, m_t$

[12]

As for an example let's say we have to encrypt 2 bits  $m_1$  and  $m_2$ .

Additive:[12]

$$\begin{aligned} EncGM(m_1) \times EncGM(m_2) &\equiv (bm_1 \times r_{12}).(bm_2 \times r_{22}) \bmod n \\ &\equiv b(m_1 + m_2)(r_1 r_2) \bmod n \\ &\equiv EncGM(m_1 \oplus m_2, pk) \end{aligned}$$

Gentry and Halevi has used a variant model similar to smart Vercauterens implementing this fully homomorphic encryption [21][22]. The smart and Vercauterens didn't have the ability to implement bootstrapping functionality in the conversion of it to a fully homomorphic encryption

- The optimization for bootstrapping functionality is a batching method is used for encryption and it is done in all aspects to implement fully homomorphic encryption. In somewhat homomorphic encryption (SWHME), it does not need a full polynomial inversion when generating the key[8].[15]

The FHE schemes has many schemes[15]. Most famous and common schemes are

- Ideal lattice based scheme,
- Integer based scheme,
- Ring learning with error,
- Elliptic curve cryptography based scheme.

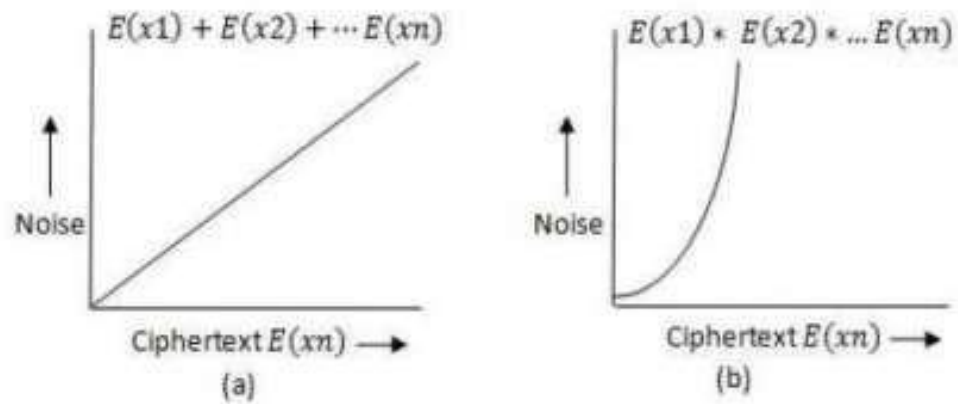
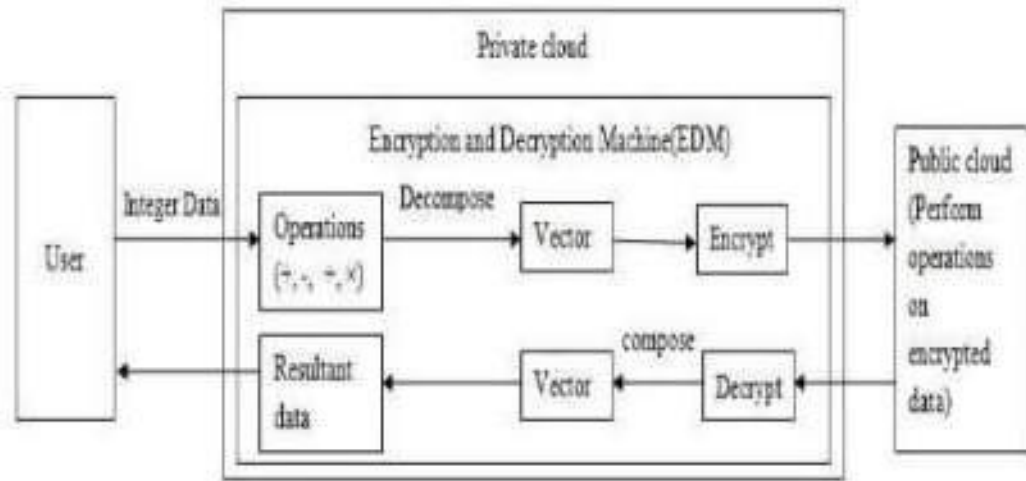


Diagram for noise expansion with encryptions [15]



Analysis and Comparison of Various Fully Homomorphic Encryption[15]

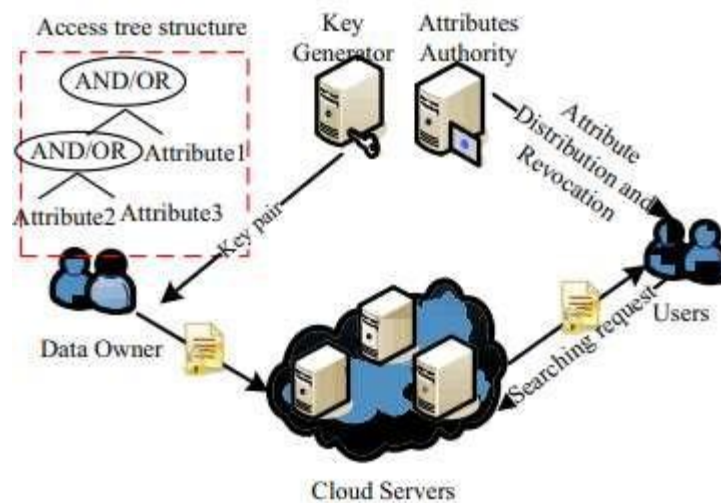
In general following table represents the comparison those encryption schemes which studied in [15].

| Property           | Ideal lattice based                             | Ring learning with error | Integer based                           | Elliptic curve cryptography based                                                                        |
|--------------------|-------------------------------------------------|--------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------|
| Introduced in Year | 2019                                            | 2010                     | 2011                                    | 2016                                                                                                     |
| Hardness           | Closet vector problem,sparse subset sum problem | Approximate GCD problem  | Rings learning with errors              | Discrete logarithm of an arbitrary eelptic curve component concerning to known base point is impractical |
| Security           | More                                            | Less                     | More than ideal based                   | More                                                                                                     |
| Simplicity         | Less                                            | More                     | Simpler than ideal based                | More                                                                                                     |
| Limitations        | Public key is too large                         | Reduced security         | NA                                      | Does not support float point calculation                                                                 |
| Efficiency         | efficient                                       | Least efficient          | More efficient that ideal lattice model | Most efficient                                                                                           |

## Use Cases

FHE is increasingly seen as a solution for privacy-preserving data processing in cloud environments, enabling secure computations on encrypted data. However, FHE remains computationally intensive and thus challenging to deploy widely in real-time applications.[1]

As to searching over encrypted data, the most common natural application of these encryption is to search data over encrypted data filed. Even though most systems supported searching over encrypted data are using bilinear maps[24][25] and those systems are not supported by multiple keys while having a low efficiency.[16]



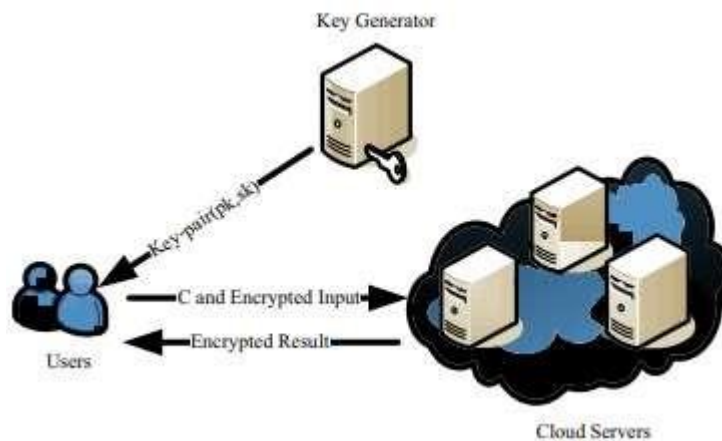
[16]System Models for encrypting searching.

## Delegation of computation

Another typical application is delegation of computation[9]. A number of developments are fueling the growing trend of "outsourcing" computing from a less capable device—like smartphones—to a more capable computational service[16].

Even if using this method of operation offers so many benefits, there still pose a serious threat to privacy and security[15][16]. The primary disadvantage is the fear of integrity and secrecy of data. The user can remove fear by encrypting their data, however on the other hand an additional factor that contributes to certain cloud computing issues. FHE permits a service to execute calculations on the data that is

encrypted. As seen in Figure 2, there is a fundamental framework for computation delegation using FHE.[16]



[16]

**Somewhat Homomorphic Encryption (SWHE)** : This introduce to process more that one process-multiplication and addition, but the number if process permitted are limited.[ns4]

## 2.5.4 Key points for FHE [17]

### 1. Confidentiality in FHE

- **Confidentiality:** One of the core principles of FHE is that it ensures strong confidentiality, meaning that an attacker cannot learn anything from the encrypted data without having the secret key (or decryption key). In other words, no information from the encrypted data (ciphertext) can be revealed—each bit remains hidden[3],[1],[17].
- **Malleability in FHE:** On the other hand, FHE allows "homomorphic" operations (e.g., addition, multiplication, bit operations such as XOR ,OR etc.) to be performed on encrypted data. This malleability opens a door to a specific type of attack, where an attacker could manipulate (or "flip") bits of the encrypted result without knowing the content of the data. Think of it like tampering with an encrypted calculation—while the attacker can't read the data, they can still make changes to it.

This could be likened to **side-channel attacks**, where an attacker can manipulate or tamper with a system's physical execution (like with a "magical laser beam" targeting specific bits), impacting the outcome. This is dangerous because if computations are tampered with, the decrypted result might be incorrect even though the attacker never saw the data.[3][17]

- **Mitigation:** FHE systems prevent this by using error-correcting mechanisms such as redundancy. For example, instead of encoding a piece of information once, you could encode it multiple times, or use replication, so that any tampering can be detected or corrected.[3]

## 2. FHE Key Structure

FHE typically involves three types of keys to manage encrypted computations:

- **Decryption Key (Master Key):** This is the main key of the cryptosystem. It is highly secure and should never be shared or transmitted. It is the key that allows someone to decrypt the encrypted data. In a typical FHE system, only the owner of this key can decrypt the results of homomorphic computations. All remaining keys are derived from this master key, but they don't have the ability to decrypt the data themselves.[3]
- **Encryption Key:** This is a public key used for encryption. Anyone who wants to encrypt data and send it to the decryption key owner uses this key. In FHE, this key might involve random encryptions of zero. While this may sound odd, it works because FHE allows for arithmetic operations on the encrypted data (such as adding or multiplying). With these encrypted zeros, you can still encrypt and manipulate any other message.[3]

**Affine Functions Support:** Since FHE allows affine functions (i.e., functions that involve adding or multiplying with constants), the encryption key can be used to encrypt any type of message even if it seems like it's just encrypting random numbers.[3]

- **Evaluation Key:** This key is given to anyone who needs to compute functions on the encrypted data. Importantly, this key does not allow someone to decrypt the ciphertext, so it's safe to distribute publicly. The size of the evaluation key depends on the complexity

of the functions. For simple linear functions (e.g., addition, multiplication by constants), the key might be small. However, for more complex operations or arbitrary functions, the key could be much larger.[3]

### 3. Why These Keys Matter in FHE:

- The encryption key allows anyone to transfer encrypted data to the owner of the decryption key.[3]
- The evaluation key allows computations on encrypted data to be outsourced to third parties (e.g., cloud providers), without those third parties learning anything about the data.
- The decryption key ensures that only the owner can read the results of those computations.

The encryption and evaluation keys enable secure computation/operations, while the decryption key ensures that the data's confidentiality remains intact in the process. This key structure allows for a separation between data encryption, computation, and decryption, facilitating a secure encrypted computation without revealing any information to unauthorized parties.[3]

#### 2.5.5 Challenges in Implementing FHE:

Although FHE has major potential for privacy and security, it comes with some significant challenges and problems[3][1]:

- **Computational Complexity:** The primary issue with FHE is its inefficiency. Performing operations on encrypted data is much slower than operating on plaintext. As for an example, a basic operation like addition or multiplication, which takes milliseconds on plaintext. But for FHE it could take seconds or longer when performed on encrypted data due to its complex mathematical process. This makes real-time applications difficult and slower, especially when the volume of data is large[3][1](results in higher time complexities). Despite the fact the recent breakthroughs that have made FHE faster, its performance still lacks some significantly behind plaintext operations. For the FHE to see wider

adoption, further research is required to reduce the resource consumptions associated with homomorphic computations.

- **Ciphertext Expansion:** FHE ciphertexts are typically much larger and longer than plaintexts. This results in significant storage consumption and increased bandwidth usage, which can be problematic in cloud-based applications. The large ciphertext size can also impact processing times since larger ciphertexts take more time to compute on and process.

Current FHE schemes like the Brakerski-Gentry-Vaikuntanathan (BGV) scheme or the Fan-Vercauteren (FV) scheme are working to reduce the expansion and time consumptions, but still present notable overheads and issues. Solving this challenge is essential to making FHE practical for data-intensive applications like big data analytics or machine learning in future areas.[3][1][2]

## **2.5.6 Applications of Homomorphic Encryption**

### **1. Cloud Computing**

One of the most promising and emerging applications of homomorphic encryption is in cloud computing sector. Cloud services are increasingly used to store and process sensitive data, but this presents security and privacy challenges, as data must be encrypted to prevent unauthorized access. However, traditional encryption requires decryption before data can be processed, which creates vulnerabilities. HE, particularly FHE, allows encrypted data to be processed without ever exposing it, thus maintaining data privacy and security.[3]

**Use Case:** In the healthcare industry, homomorphic encryption could enable cloud-based processing of patient records, allowing data analysis for medical research without compromising patient confidentiality.

### **2. Privacy-Preserving Machine Learning**

Machine learning models typically require large amounts of data to be trained. Privacy concerns arise when this data includes personal or sensitive information. Homomorphic encryption

enables organizations [3][20] to perform machine learning tasks on encrypted data, thus protecting privacy without sacrificing the benefits of collaborative or distributed learning.

**Use Case:** Two hospitals can collaboratively train a new machine learning model on from their combined total datasets, without exposing the sensitive patient data to one another, with the help of HE.

### 3. Blockchain and Cryptocurrencies

In a decentralized systems like blockchains, maintaining both transparency and privacy is really important. Homomorphic encryption can add a layer of privacy to transactions, which allowing participants to process and verify the encrypted data without revealing the actual transaction details to a third party. This would enhance the privacy of users while still allowing for necessary validation processes in secure manner.[3]

**Use Case:** In cryptocurrency, homomorphic encryption could be used to allow private transactions while still providing a way for miners to verify the correctness of the transaction without exposing its details to others.[3][1]

## 3. Future Developments in Homomorphic Encryption

### 3.1. Performance Optimization

Homomorphic encryption (HE) has historically been seen as computationally expensive and has higher resource consumptions, due to its intensive and complex mathematical operations. Researchers are working on improving the efficiency of HE algorithms to make it more practical for real-world applications, while maintaining the security aspects provided by the this encryption.[3]

- **Batch Processing:** This technique allows multiple data items to be encrypted and processed simultaneously, reducing the overall computational load. Instead of performing homomorphic operations on single ciphertexts, batch processing handles several ciphertexts at once, significantly improving throughput.[3][1]
- **Optimized Data Structures:** Advancements in the underlying data structures, such as optimized polynomial representations and ciphertext packing, help reduce the size of

encrypted data and the time needed to perform operations. Efficient algorithms are being developed to perform these operations faster, ensuring that FHE can handle real-time applications, such as financial transactions or healthcare data analysis.[1][5][3]

- **Parallel Processing:** The use of parallel computing and GPUs can further accelerate homomorphic operations by distributing computational tasks across multiple processors, thus reducing time complexity.[1][6][3]

### 3.2. Hybrid Encryption Models:

In future applications, we may see FHE combined with other cryptographic techniques like **Multi-Party Computation (MPC)** and **Secret Sharing**. These hybrid models aim to leverage the strengths of each encryption method to improve both security and performance. In fact, some algorithms are already developed and proposed already.

- **MPC + FHE:** Combining FHE with MPC allows multiple groups to compute or to perform calculations on encrypted data without revealing their individual inputs. This hybrid approach would optimize the overall computation and communication costs, making secure data processing more efficient.[1][8][3]
- **Scalability:** Hybrid models are expected to scale better than pure FHE. For instance, in distributed systems like cloud computing, a combination of FHE and other techniques may provide secure, privacy-preserving computations that are efficient enough to scale to large datasets and multiple users.[1][8][3]

### 3.3. Post-Quantum Security

Quantum computers pose a threat to many classical cryptographic schemes, including RSA and ECC, as they are susceptible to attacks from quantum algorithms like Shor's algorithm. To counter this, cryptographic research is focusing on **post-quantum cryptography** to ensure the long-term security of encryption schemes:[3][2][5][9]

- **Lattice-Based Cryptography:** Most FHE schemes today are based on lattice problems, which are believed to be resistant to quantum attacks. Lattices offer security based on the hardness of specific mathematical problems, such as the Shortest Vector Problem (SVP)

and Learning with Errors (LWE), which remain still difficult to solve even for quantum computers, due its complexity in mathematical perspective ensuring that its difficulty in reverse engineering without any help.[1][3][9]

- **Quantum-Resistant FHE:** Enhancing FHE schemes with quantum resistance is crucial for protecting data in a post-quantum world. Current research is investigating more robust lattice-based constructions and their ability to withstand quantum decryption attempts.

### 3.4. Wider Adoption in Healthcare and Finance:

As technology evolves in modern era, these sectors are likely to adopt FHE like encryptions to enable secure data analytics and processing while maintaining privacy:[1][5][9][3]

- **Healthcare:** Medical data is sensitive and protected by stringent privacy regulations like HIPAA rules and regulations. FHE could allow researchers and healthcare providers to analyze encrypted patient data for clinical insights or diagnostical decisions and machine learning models without exposing private information of patients. This would enable a large-scale medical research on encrypted datasets.[5][16][3]
- **Finance:** With constant concerns about privacy and regulatory compliance, the finance sector is highly data-sensitive. FHE could allow financial institutions to run risk assessments, fraud detection algorithms, or financial modeling on encrypted transaction data, ensuring privacy without compromising data utility and data breaches /leakages.[3][5][9][20]
- **Regulatory Compliance:** Privacy-preserving techniques like FHE will play an essential role in allowing organizations to meet global data protection regulations, such as the General Data Protection Regulation (GDPR), by ensuring secure computation and operations on encrypted data.[3][5][20]

## 4. Conclusion

Homomorphic encryption shows a major breakthrough in the field of cryptography, providing a method to process data securely without exposing it to the other world. From its theoretical perspectives and origins to recent advancements in efficiency, it has the potential to transform data privacy in cloud computing, secure machine learning, and beyond. The major findings mentioned in this report show the importance of FHE, which is protecting sensitive data and its relevance in today's data-driven technological world. Future developments, including such as performance optimizations, hybrid models, post-quantum security, and industry adoption, will continue to enhance the area of homomorphic encryption, making it a major factor of future cybersecurity solutions. As researchers are in progress, homomorphic encryption will most likely to become a major part of the broader security infrastructure, providing unordered levels of data privacy and protection.

Furthermore homomorphic encryption (HE) highlight its significant potential and transformative impact in various fields.

- **Advancement in Data Security:** By providing a secure method for performing computations on encrypted data, it has supported to be a crucial innovation. For enhancing data security and privacy, especially in sensitive areas like healthcare and finance, this ability/capability is important. [3][1][5]
- **Real-World Applicability:** Many researchers have validated the practical applications of HE through simulations and analyses. It shows how that HE techniques are efficient, scalable, and secure, making them applicable for real-world scenarios. Such as secure computation outsourcing and privacy-preserving data analysis like areas.[3][5][9].
- **Call for Adoption:** Many researchers, scientists and computer scientists are advocate for the widespread adoption and integration of HE technologies. Because that as cybersecurity threats evolve and the amount of sensitive data increases, HE can play a major role in ensuring data confidentiality and integrity [3][5][9]

- **Future of Data Security:** In future many researches predicted that ,HE could revolutionize how sensitive data is handled, promoting a more secure and privacy-conscious approach in an interconnected world [1][5][9][22]

With the increasing reliance on cloud services for data storage and processing, the need for robust security measures has never been more critical than ever. These sensitive data which is stored in the cloud , is vulnerable to unauthorized access and breaches, highlights the importance encryption for protecting personal information , which highlights the importance of data security[3][14][21].

Since Homomorphic encryption allows operations to be performed on encrypted data without needing to decrypt it first, meaning that sensitive information can remain secure while still being usable for various computations, it is a game-changer for industries that handle confidential data [3][14][15][19].

Understanding the metrics is important for developers and organizations when choosing the appropriate encryption scheme for their needs. Smaller key and ciphertext sizes can lead to improved performance and reduced storage requirements, which are essential factors in cloud environments [22][14][3].

Future work could focus on enhancing the efficiency of these encryption methods, since it needed high processing power and consume resources for processing , exploring new efficient algorithms, and addressing the challenges which are associated with fully homomorphic encryption, which allows for more complex operations on encrypted data [3][21][11].

Homomorphic encryption methods provides valuable insights into the balance between security and functionality of areas such as in cloud computing. As data breaches continue to make significant risks, the adoption of effective encryption techniques will be crucial for safeguarding sensitive information in this modern digital age. The ongoing research in this area will likely lead to more advanced solutions that can further enhance data security while maintaining usability and performance [3][24].

Transformative power of homomorphic encryption, highlights its integration into secure computation practices to address the contemporary challenges in data security and privacy. And

Even though this topic was slightly silent in past decade , with new technology this opens new doors for post quantum cryptography and protection from quantum attacks.

## 5. References

- [1] Cryptology ePrint Archive, The Cryptography Caffè, available at <https://eprint.iacr.org/>.
- [2] Kamble, A., Jiet, M. M., & Puri, C. (2024). *Homomorphic Encryption and its Applications in Multi-Cloud Security*. <https://doi.org/10.1109/iciict60155.2024.10544773>
- [3] Craig, Gentry. (2009). Fully homomorphic encryption using ideal lattices. 169-178.  
Available from: 10.1145/1536414.1536440
- [4] Online resource: - [Learning with Errors and Ring-LWE in FHE](#)
- [5] Online resource: - [Practical Applications of Homomorphic Encryption in Cloud](#)
- [6] Online resource: - [Privacy and Security in Cloud Computing with Homomorphic Encryption](#)
- [7] Online resource: - [Challenges in Fully Homomorphic Encryption Implementations](#)
- [8] Online resource: - [Ciphertext Expansion and Its Impact](#)
- [9] Online resource: - [Use of Homomorphic Encryption in Privacy-Preserving Machine Learning](#)
- [10] Online resource: - [Blockchain and Cryptocurrency Enhancements with HE](#)
- [11] Online resource: - [Performance Optimizations in HE Systems](#)
- [12] Online resource: - [Future of Quantum-Resistant Cryptographic Schemes](#)
- [13] Zainab, H., Mahmood., Mahmood, Khalel, Ibrahim. (2018). 1. New Fully Homomorphic Encryption Scheme Based on Multistage Partial Homomorphic Encryption Applied in Cloud Computing. Available from: 10.1109/AICIS.2018.00043.
- [14] (2022). 1. Performance Evaluation of RSA, ElGamal, and Paillier Partial Homomorphic Encryption Algorithms. Available from: 10.1109/csase51777.2022.9759825
- [15] P. Chaudhary, R. Gupta, A. Singh and P. Majumder, "Analysis and Comparison of Various Fully Homomorphic Encryption Techniques," *2019 International Conference on Computing, Power and Communication Technologies (GUCON)*, New Delhi, India, 2019, pp. 58-62.
- [16] L. Cardoso dos Santos, G. Rodrigues Bilar and F. Dacêncio Pereira, "Implementation of the fully homomorphic encryption scheme over integers with shorter keys," *2015 7th International*

*Conference on New Technologies, Mobility and Security (NTMS)*, Paris, France, 2015, pp. 1-5, doi: 10.1109/NTMS.2015.7266495 [17] Jonathan Katz and Yehuda Lindell, "Introduction to Modern Cryptography," CRC Press, 2021.

[18] Kadykov, Victor & Levina, Alla & Grebenevich, Ksenia. (2024). Implementation of Lattices System in Homomorphic Encryption. 1-5. 10.1109/ACDSA59508.2024.10467404

[19] R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Commun. ACM*, vol. 21, no. 2, 1978, doi: 10.1145/359340.359342.

[20] M. Derfouf and M. Eleuldj, "Cloud Secured Protocol based on Partial Homomorphic Encryptions," *2018 4th International Conference on Cloud Computing Technologies and Applications (Cloudtech)*, Brussels, Belgium, 2018, pp. 1-6, doi: 10.1109/CloudTech.2018.8713353.

[21] Ogburn, Monique & Turner, Claude & Dahal, Pushkar. (2013). Homomorphic Encryption. *Procedia Computer Science*. 20. 502-509. 10.1016/j.procs.2013.09.310.

[22] Khalid El Makkaoui ; Abderrahim Beni-Hssane ; Abdellah Ezzati, "Can hybrid Homomorphic Encryption schemes be practical?," in *International Conference on Multimedia Computing and Systems (ICMCS)*, Marrakech, Morocco, 2017.

[23] K. B. Logunleko, O. D. Adeniji, and A. M. Logunleko, "A Comparative Study of Symmetric Cryptography Mechanism on DES, AES and EB64 for Information Security," *Int. J. Sci. Res. Comput. Sci. Eng.*, vol. 8, no. 1, 2020.

[24] I. H. Latif, "Time Evaluation of Different Cryptography Algorithms Using Labview," in *IOP Conference Series: Materials Science and Engineering*, 2020, vol. 745, no. 1, doi: 10.1088/1757-899X/745/1/012039

[25] Homomorphic Encryption for Secure Cloud Computing Kaledio Potter, Dylan Stilinki and Selorm Adablanu. doi: DOI: 10.13140/RG.2.2.19574.41285